An Experimental Evaluation of Keywords-aware Optimal Routes on Spatial Database

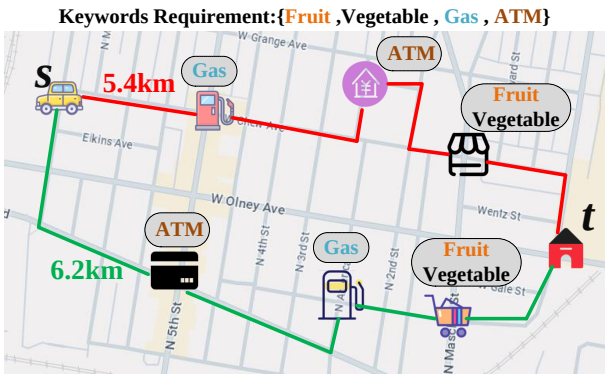

Fig. 1. Routes Satisfying the User Needs

***Abstract*—As more detailed POI (Point of Interest) information has been incorporated into road network, routing has evolved from finding paths from one place to another, to satisfying users' needs (keywords) along the trip. However, the existing solutions either only support one keyword per POI, or require a fixed visiting order, or only provide one option to choose from. Therefore, we study the *top-k Optimal Routes with Collective Spatial Keywords* (k*-ORCSK*) problem, which is the most general keyword-aware routing problem that supports multiple keywords, arbitrary orders, and top-k results. To solve this problem, we apply an enumeration framework and reduce the complexity by contracting non POI-related vertices and taking the keywords into account. After that, we propose a best-first path expansion method *DA-CSK* based on deviation to convert the enumeration paradigm from the distance-oriented to the keyword-oriented.**

I. Introduction

The proliferation of urban GPS sensing devices, location-based social networks, and spatial databases [1] has led to a surge in the number of route planning query requests that cater to various user needs within cities. Nowadays, route planning can accommodate user-specific requirements, such as querying for the shortest path, the fastest path, multi-objective optimized paths, multi-modal transportation paths, and spatial keywords-aware routing, among others. Among these shortest path problems, spatial keywords-aware route planning [2]–[5] is most common in real life. When users plan a trip, they often have one or several keywords they wish to visit to fulfill their needs. Therefore, spatial keywords-aware route planning is to devise an optimal route for the user that encompasses all queried keywords information. In road networks, keywords information is usually found within POI(Point of Interest). Thus, a POI might contain single or multiple keywords. Furthermore, users may access POI in either a fixed or arbitrary sequence.

Take Figure 1 as an example, an user drives from start point s to home t, buy some fruits and vegetables, deposit money in bank, and fuel his car. We decompose the user's needs into four keywords: fruits, vegetables, gas, and bank. Based on the locations and the associated keywords of the POIs, Both the green route and the red route can meet the user's needs, with route lengths of 6.2 km and 5.4 km, respectively. If user wants to reach the destination as early as possible, clearly, we should recommend the red route because it has the shortest distance.

We can refer to the above-mentioned problem as the ***S**patial **K**eywords-aware **O**ptimal **R**outing **P**roblem*(*SKORP*). In the past decade, various algorithms have been proposed to address the *SKORP*. They can be divided into two categories: those based on POI candidate sets and those based on path expansion. In this paper, we conduct a comprehensive experimental evaluation of the algorithms across multiple dimensions. This can further guide the selection of the appropriate algorithm for different application scenarios.

A. Motivation

1) ***Filling the gap in algorithm evaluation****:* There is a review paper [6] on the *SKORP* problem. However, it only summarizes problem definitions and algorithms. There is no experimental evaluation of the algorithms. Since the last review, many new algorithms have been developed. Therefore, we need to consider them and conduct a comprehensive experimental evaluation.

2) ***Inconsistency in Datasets****:* To truly evaluate the potential of each algorithm, it is necessary to use datasets of varying scales. However, existing algorithms have not been compared using the same dataset, and the data scales vary. For example, the *GOD* algorithm [4] used the Odbg and Beijing road networks, while the *MRRP* [2] algorithm utilized the Tainan and New York road networks. However, the number of points in New York is three times that of Beijing. Therefore, we use a unified dataset of different scales for algorithm evaluation.

3) ***Standardized algorithm implementation****:* The existing algorithms are implemented in either C++ [2], [3], [5], [7] or Java [4], and the experimentation is conducted on various operating system platforms, including Windows [4] and Linux [2], [3], [7]. To conduct a comprehensive experimental evaluation, it is essential to use a standardized implementation of the algorithms. The potential impact of different data structures, programming languages, and hardware performance on the efficiency of the algorithms cannot be ignored.

*4) **Unified Optimization Technique**:* Through experiments, we find that certain operations within the algorithms may significantly impact the overall performance. For example, the algorithm requires multiple iterations of shortest distance queries and nearest neighbor queries. Dijkstra's [8], A* [9], Contraction Hierarchies (CH) [10], and H2H [11] techniques all support shortest distance queries, but H2H is approximately five orders of magnitude faster than Dijkstra's algorithm. Therefore, employing a unified optimization technique to enhance all algorithms before conducting experimental evaluations is imperative.

B. Contributions

The contribution of this paper is as follows:

- We conducted a comprehensive experimental evaluation of eight algorithms(*KNN-MS,KMD-MS,EMB* [2], *GOD* [4], *IG-Tree* [3], *PruningKOSR, StarKOSR* [7]and *DA-Prune* [5]), considering every factor that could influence their performance. These factors include preprocessing time, index size, road network scale, POI density, POI distribution, keyword variety, query request size, and OD distance.
- We employ a unified optimization technique to further enhance common operations in these algorithms, facilitating fair competition on an equal footing. For frequent distance queries within the algorithms, we replace them with the state of art algorithm H2H [11]. Additionally, for the priority queues in some of the algorithms, we used a binary heap, and for storing keyword information in POIs, we employed binary bit storage. This approach further enhances the processing efficiency of the algorithms.
- We further identified the applicable scenarios for each algorithm under the influence of different parameters. The purpose of this approach is to enable the selection of the most suitable algorithm for any given road network, regardless of its scale, based on the current experimental evaluation results. The road network data can be obtained from [12], and all source code in the paper has been categorized by module and made publicly available at [13].

II. Related work

In this section, we compare the related works from the following three dimensions: POI order, number of keywords, and the format of path results.

A. Fixed / Arbitrary POI Visit Order

As for the fixed order query,*OSR* [14] is the first to consider POI in shortest path queries. It proposes the *LOAD*, *R-LOAD(extend LOAD with R-tree)*, and *PNE* algorithms, which are related to the classical Traveling Salesman Problem (*TSP*) [15]. However, the *OSR* algorithm supports Euclidean space and therefore cannot be directly applied to road networks (road network distances are typically greater than Euclidean distances). Similarly, the *KOSR* [7] also defines a fixed POI access order and proposes two alogrithms to solve *KORP* :*PruningKOSR* uses the dominance relationship to filter temporarily unnecessary routes and *StarKOSR* uses the target-directed strategy to find the optimal feasible routes.

As for the arbitrary visiting order, typically results in a further expansion of the search space, leading to the generation of more POI combinations. MRRP(*Multi-Request Route Planning*) [2] solves *SKORP* with four algorithms (*KNN-MS, KMD-MS, EMD, KRA-MS*).The core idea of *MRRP* is to utilize nearest neighbor queries to acquire a candidate set of POI and construct path. The GOD [4] algorithm also supports *KORP* and use grid index to accelerate the generation of POI candidate sets and consider the distribution of query keywords within the road network. IG-Tree [3] partitions the road network to store POI information to facilitate the generation of POI candidate sets. DA-Prune [5] adopts vertex offset strategies and constructs paths using heuristic functions.

In this paper, to increase the generality of the problem, we only consider scenarios where POIs are visited in arbitrary visiting order.

B. Single / Multiple Keyword in Each POI

Most existing works [**?**], [**?**], [**?**], [**?**], [**?**], [**?**], [16]–[18] assume that each POI contains only one keyword. However, in real-world scenarios, a single POI often covers multiple keywords. For example, a shopping mall may cater to eating, watching movies, and buying clothes. *MRRP* [2], *IG-Tree* [3], *GOD* [4], and *DA-Prune* [5] algorithms all support multi-keyword queries.

In this paper, we consider scenarios where each POI covers multiple keywords.

C. One / multiple shortest paths

Typically, users select one optimal path as the result, but in certain scenarios, providing multiple paths may be preferable for users. For example, Skyline paths return a set of non-dominated paths across multiple dimensions, while KOSR [7] and DA-Prune [5] return the top-k paths for user selection. Group queries [16]–[18] return the top-k routes that minimize the sum of paths for all members of the group. .In this paper, we only consider returning one optimal path (Top-1).

III. Problem definition

In this section,we define the problem definition for all algorithms within experimental evaluation and the corresponding concepts formally.

A road network is a graph $G(V, E)$, where V is a set of vertices and $E \subseteq V \times V$ is a set of edges with each edge $(u, v) \in E$ having a weight $w(u, v)$. A path p is a sequence of vertices $\langle v_0, \ldots, v_n \rangle$ where $(v_i, v_{i+1}) \in E, \forall i \in [0, n-1]$. The distance of p is denoted as $\omega(p) = \sum_{i=0}^{n-1} w(v_i, v_{i+1})$. Each vertex $v_i \in V$ contains zero or multiple keywords. For the rest of this paper, we denote the vertex that contains keyword as POI. Let $K(\cdot)$ be the union of keywords of a set of vertices. For instance, $K(\{v_i\}) = \{kw_1, \ldots, kw_l\}$ denotes the keyword sets of POI v_i. Note that the lengths of keyword set could be different for different POIs. Similarly, the keywords set

TABLE I
IMPORTANT NOTATIONS.

Notation	Definition
$G = (V, E)$	Road network with vertex V and edge E
n, m	The number of vertices and edges
$p = \langle v_1, ..., v_k \rangle$, $\omega(p)$	A path from v_1 to v_k and the sum distance
$sp(v_i, v_j)$, $\delta(v_i, v_j)$	Shortest path from v_i to v_j and its distance
$K(\cdot)$	Keyword set of vertices or a path
$PSR(kw_i)$	POI set containing the Request(Keyword) kw_i
Q_{kw}	User-specified keywords set of query Q
$RSP(p)$	Requests(Keywords) served in POI p
C_{POI}	Candidate set of POIs,$C_{POI} \subseteq PSR(Q_{kw})$
R_{p_i}	POI results set of the valid path p_i

Information	
POIs	**keywords**
v_2	kw_1
v_3	kw_2
v_5	kw_1 kw_2
v_7	kw_3
v_8	kw_2
v_{10}	kw_1 kw_3
v_{11}	kw_2 kw_3

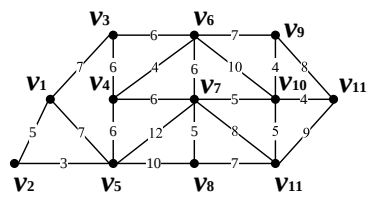

Fig. 2. Example Road Network with POIs and Keywords

of a path p is notated as $K(p)$. Moreover, we maintain an inverted list $RSP(kw_i)$ for each type of keyword kw_i to store the vertices that contain kw_i and $PSR(Q_{kw})$ to containing required keywords in Q_{kw}. We summarize the notations used in this paper in TableI.

Figure 2 shows an example, where we have the whole keywords set $K(V) = \{kw_1, kw_2, kw_3\}$, and $O(kw_1) = \{v_2, v_3, v_7\}$. Consider a path $p_1 = \langle v_1, v_2, v_3, v_6, v_9 \rangle$, We have its keywords set $K(p_1) = \{kw_1, kw_2\}$ as $K(\{v_2\}) = \{kw_1\}$, $K(\{v_3\}) = \{kw_1, kw_2\}$, and $K(\{v_6\}) = \{kw_2\}$. Next, we define the collective keyword covering route as follows.

Definition 1 (Partial Valid Path). *Given a path p, if keywords set of p contains user-specified keywords set Q_{kw}, i.e., $(K(p) \cap Q_{kw}) \subseteq Q_{kw}$.We define path p as a Partial Valid Path.*

Definition 2 (Valid path). *Given a set Q_{kw} and a partial valid path p, p covers Q_{kw} if the keyword set of p contains Q_{kw}, i.e., $Q_{kw} \subseteq K(p)$. We define p as a keywords-aware valid path w.r.t. Q_{kw}.*

Given a pair of source and destination vertices and a set of keywords, we can retrieve multiple valid paths.Accordingly, the query that finds the shortest valid path is defined as follows.

Definition 3 (Spatial Keywords-aware Optimal Routing Problem(*SKORP*)). *Given a road network $G(V, E)$, a starting vertex s, a destination t, and a set of keywords $Q_{kw} \subseteq K(V)$, let P be all possible valid paths w.r.t Q_{kw} from s to t. The SKORP query is to return $p \in P$ and the corresponding set of POIs, R_p, such that $\omega(p) \leq \omega(p')$, for all $p' \in P \setminus p$.*

SKORP is NP-hard When each POI is associated with a unique keyword, and each keyword is exclusive to a single POI, the *SKORP* is reduced to the NP-hard Traveling Salesman Problem (*TSP*) as proved in [15].So if a POI is associated multiple keywords and each keywords is exclusive to more than a single POI, *SKORP* also a NP-hard Problem.

IV. SKORP ALGORITHM

In this section, we classify the algorithms into two major categories based on the method of path generation. The first category involves first obtaining a set of POI candidates and then constructing the path. The second category begins at the start point s and constructs the path through a path expansion strategy.We will introduce algorithms belonging to these two categories separately.

A. Candidate Set based

Given a set of user specified keywords Q_{kw}, it is certain that there are numerous POIs in the road network, each of POI can satisfy at least one keyword from Q_{kw}. A direct approach is to obtain all combinations of POIs that satisfy Q_{kw} . However, to gather all possible combinations of POIs would inevitably lead to a huge search space, which is impractical in a real-world road network. Therefore, it is necessary to use efficient strategies for generating candidate sets of POI in order to further reduce the search space and enhance efficiency.

1) Direct Minimun Distance generate strategy: Direct Minimun Distance (DMD) generate strategy is proposed by [2]. The *DMD* strategy considers the direct minium distance $\delta^D(s, p, d)$ of each POI p which containing the keyword in Q_{kw}, *DMD* is calculated by:

$$\delta^D(s, p, d) = \delta(s, p) + \delta(p, t) \tag{1}$$

Point s is the user-specified starting point, and t is the destination. The purpose of calculating *DMD* is to ensure that the POIs are closer to the destination t, rather than far from it. According to *DMD* strategy, the *KMD-MS* algorithm is proposed by [2].

The basic idea of *KMD-MS* is as follows : For each keyword kw in Q_{kw}, it computes k POIs with minimal *DMD* by $PSR(kw_i)$. Subsequently, it selects a POI that can satisfy the most keywords to be added to the C_{POI}. Additionally, it is worth noting that there might be duplicate POIs in the C_{POI}, because a POI can be the *DMD* result for multiple keywords.

For the the query $Q(v_1, v_{12}, \{kw_1, kw_2, kw_3\})$ in Fig 2, and k=2. We can obtain POI sets for each keywords: $PSR(kw_1)$=$\{v_2, v_5, v_9\}$,$PSR(kw_2)$=$\{v_3, v_8, v_{11}\}$, and $PSR(kw_3)$=$\{v_6, v_9, v_{11}\}$. Next, we calculate the k POI points with the smallest *DMD* within each sets.For kw_1 are v_5 and v_9;for kw_2 are v_3 and v_8;for kw_3 are v_6 and v_9.Finally,select the POIs that can satisfy the most keywords from each sets,get $C_{POI} = \{v_3, v_9\}$.

2) Embed-Based generate strategy: If the selected POIs are near to or on the shortest path from start point s to destination t the route distance would decrease. This is clearly understandable: if the POIs near the $sp(s, t)$ already fully satisfy the keywords user-need, then we only need to find an optimal path that includes these POIs. Therefore, the *EMB* algorithm [2] employs this idea to retrieve candidate sets and construct paths.

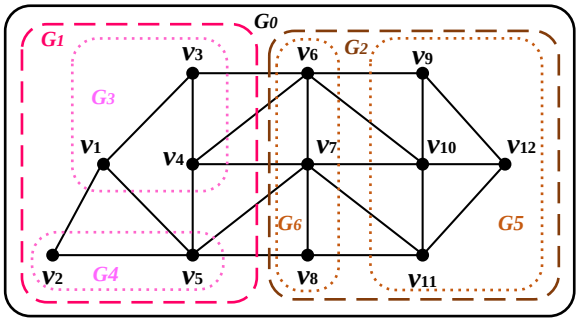

Fig. 3. IG-Tree example

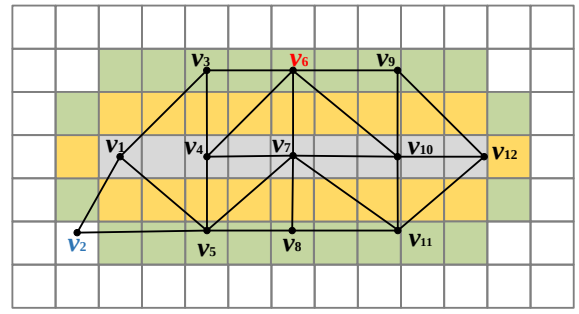

Fig. 4. Grid Index example

The main idea of *EMB* is as follows: First, use the shortest path algorithm to find the shortest path from s to t, denoted as δ_{EMB} and to obtain the keywords set $K(\delta_{EMB})$. Then, obtain keywords which can be served by δ_{EMB} from Q_{kw}. For keywords that have not yet been satisfied by $K(\delta_{EMB})$, denoted as $Q_{kw}^{rem} = \{kw_1^{rem}, \ldots, kw_m^{rem}\}$. *EMB* get a POI p^* with the minimum *DMD* $\delta^D(s, p^*, d)$ from $PSR(kw)$ for each keywords kw in Q_{kw}^{rem} . Whenever a new POI is found, it needs to be inserted into $sp(s,t)$ until the path, after insertion, satisfies Q_{kw}. The insertion strategy is to find a position of δ_{EMB} such that *additional moving distance (AMD)* is minimized, AMD is shown as (2).

$$AMD = \delta(p_j, p^*) + \delta(p^*, p_{j+1}) - \delta(p_j, p_{j+1}) \qquad (2)$$

For the the query $Q(v_1, v_{12}, \{kw_1, kw_2, kw_3\})$ in Fig 2. First find the shortest path $sp(v_1, v_{12}) = \{v_1, v_5, v_4, v_7, v_{10}, v_{12}\}$. Because v_5 can satisfy kw_1, so we only consider kw_2 and kw_3. For kw_2, v_3 is the POI with minimum *DMD*. For kw_3,v_6 is another one. According to the *AMD*,v_3 should be inserted between v_5 and v_4, v_6 should be inserted between v_4 and v_7. Finally path is $\{v_1, v_5, v_3, v_4, v_6, v_7, v_{10}, v_{12}\}$

3) IG-Tree generate strategy: . *IG-Tree* [3] as a varsion of *G-Tree* [19], [20], by integrating keyword considerations, can effectively obtain candidate set C_{POI}. In *IG-Tree*, keywords information is stored in inverted lists at partition nodes, and distances between partition boundaries are also precomputed.

The main idea of *IG-Tree* is as follows: *IG-Tree* algorithm first determines their Minimum Common Ancestor LCA in the *IG-Tree*, denoted as $LCA(s,t)$. Based on the inverted list in partition nodes $LCA(s,t)$, we can check whether all the keywords user-need have been satisfied. If the POIs in $LCA(s,t)$ do not satisfy all user-need keywords,the search ascends to the parent node of $LCA(s,t)$ and tests the keywords iteratively until finding a partition node that contains all user-need keywords. After that, POI nodes containing the user-need keywords are retrieved from the all leaf nodes of this partition node. This procedure is conducted to obtain the candidate set of POI C_{POI}. It can be observed that the *IG-Tree* intuitively narrows the search space by finding the LCA, instead of searching across the entire road network.

Fig 3 shows the partition of road network by *IG-Tree*. For the query $Q(v_6, v_{12}, \{kw_1, kw_2, kw_3\})$. First find the $LCA(v_6, v_{12})$, v_6 is in partition G_5, v_{12} is in partition G_6, its parents are both G_0, so $LCA(v_6, v_{12}) = G_0$, and G_0 partition can satisfy all user-need keywords.Therefore, all POIs in G_0 that can satisfy query keywords are C_{POI}, $C_{POI} = \{v_6, v_8, v_9, v_{11}\}$.

4) Grid expansion generate strategy: Grid Index [21] is a index structure that divides the road network into multiple grids. Through this division, the road network is segmented into multiple rectangles, each of which stores the internal road network nodes and keyword information. *GOD* algorithm [4] uses Grid Index and considers the overall distribution of POI to obtain C_{POI}. It considers if some needed keywords can be satisfied only by one node, denoted as node p_{def}, this node should be regarded as a key point, who will definitely be visited.

The main idea of *GOD* is as follows: To obtain C_{POI}, Grid Index is used to perform two rounds of expansion. The first round of expansion aims to acquire the key point p_{def}, while the second round of expansion is to find the complete C_{POI} that satisfies Q_{kw}.

The first round of grid expansion in the *GOD* algorithm consists of two steps. The first step involves obtaining POIs, that are located within the grids that are traversed by the straight line between the s and the t. If first step does not satisfy Q_{kw}, In second step, the grids will be expanded level by level until the keywords union of all the grids can cover the Q_{kw}. Once all POIs have been collected, for each keyword kw in Q_{kw}, if there is only one POI that can satisfy kw, then this POI is regard as p_{def}. After the first round of expansion, if Q_{kw} is not satisfied, we run the second round of grid expansion. For the set of unsatisfied keywords $Q_{kw}^{rem} = Q_{kw} \setminus (K(p_{def}) \cap Q_{kw})$. we expand the grids through which the two lines pass between the s to p_{def} and from p_{def} to the t. If POIs in these grids can satisfy Q_{kw}^{rem}, then the grid expansion is stoped. Finally, p_{def} and the POIs in the second round of expansion are added to the C_{POI}.

Fig 4 shows an example of Grid Index. For the query $Q(v_1, v_{12}, \{kw_2, kw_3, kw_4\})$. The gray grid represents the area traversed by the straight line between v_1 and v_{12}. This area does not meet any of the user need keywords, so it is expanded along the gray area to the yellow grids. Subsequently, it is further expanded to the green grids. At this point, the grid has covered all the query keywords, and the first round of expansion stops. We can find that only v_6 can satisfy kw_4,

therefore key point p_{def} is v_6.The second round of expansion starts from the straight lines between v_1 and v_6,and between v_6 and v_{12}, until all remaining keywords kw_2 and kw_3 are satisfied. Finally,we get $C_{POI} = \{v_2, v_3, v_5, v_9, v_{11}\}$ by Grid Index.

5) Path generation strategy based C_{POI}: After obtaining the candidate set C_{POI} ,we need to construct a optimal path using that point in C_{POI}. Existing path generation techniques fall into two categories, both based on greedy methods.

i) *Nearest Neighbor(NN) Path generation strategy*: The partial valid path initally starts from start point s, at each step, selects a POI p_{NN} from C_{POI} that is nearest to the last point v_p on the current partial valid path p=$\langle s, v_1, \ldots, v_p\rangle$. This selected POI must satisfy at least one keyword in $(Q_{kw} \setminus K(p) \cap Q_{kw})$. Then, p_{NN} is added into v_p to build a new path, and p_{NN} as the last point of new partial path. The Nearest Neighbor strategy is iteratively executed until all the keywords in Q_{kw} are satisfied.

ii) *Most Service(MS) Path generation strategy*: In *DMD*,we select a POI from k POIs that satisfies the most keywords in Q_{kw} and add to the C_{POI}. Similarly, this idea can also be utilized during the path generation phase.The partial valid path p initally starts from start point s, at each step,selects a POI P_{MS} from C_{POI}. P_{MS} can satisfy the maximum number of unsatisfied keywords set $(Q_{kw} \setminus K(p) \cap Q_{kw})$. Then,$P_{MS}$ is added to the end of current partial path p. The Most Service strategy is also iteratively executed until all the keywords in Q_{kw} are satisfied.

B. Path Expand based

Candidate Set based method can effectively find POIs that satisfy given keywords and construct paths. However, path generation strategies based on *NN* or *MS* might lead to the final route may contains the POIs located at the opposite direction to the destination.

In methods based on path expansion, we can categorize path expansion strategies into three types.

1) X-th nearest neighbor based: During the path expansion process, if we can acquire as many partial paths as possible at a minimal cost and select the best one for further expansion, then the result path will be closer to the optimal solution.

PruningKOSR [7] extends the *PNE* [14] algorithm by integrating path dominance relationships and utilizing progressive *X-th nearest neighbor* search to expand path. We first introduce the concept of path dominance relationships.

Definition 4 (**Partial Valid Path Dominance Relationship**). *Given a query keywords set Q_{kw} and two partial path p_a and p_b with the same ending vertex, we say that p_a dominates p_b w.r.t Φ, denoted as $p_a \prec_\Phi p_b$, if the following two conditions hold: 1) The keyword set that can be satisfied by p_a and p_b is the same, $\Phi = K(p_a) \cap Q_{kw} = K(p_b) \cap Q_{kw}$; 2) The path cost of p_a is no greater than p_b, $\omega(P_a) \leq \omega(P_b)$.*

If $p_a \prec_\Phi p_b$, path p_b can be pruned safely, because the any valid path extended by p_b could not be better than p_a's.The first condition guarantees that they have the same remaining unsatisfied keyword set $Q'_{kw} = Q_{kw} \setminus \Phi$, and what they have to do next is to search POIs that can satisfy the keywords in Q'_{kw}. Hence, the POIs selected by them will be the same.

The basic idea of the *PruningKOSR* algorithm is as follows: A priority queue is maintained for partial valid path and enqueue the first path $p = \langle s\rangle$. At each iteration,the route $p = \langle v_1, \ldots, v_{p-1}, v_p\rangle$ with minimal distance is examined.If it already reachs the destination,the algorithm terminates. For each path p, perform two operations , i) consider an unsatisfied keywords set $Q'_{kw} = Q_{kw} \setminus K(p)$, a new path via v_p's nearest neighbor v_{NN}, $p_1 = \langle v_1, \ldots, v_p, v_{NN}\rangle$ is generated ,v_{NN} is calculated by (3).

$$v_{NN} = \arg \min_{\forall kw_m \in Q'_{kw}} \left(\min_{\forall v_j \in PSR(kw_m)} (\delta(v_p, v_j)) \right) \quad (3)$$

ii) we generate an another path derived from p. Since the route via the X-th nearest neighbor of v_{p-1} has been generated in previous iterations. we need to find v_{p-1}'s X+1-th nearest neighbor in $(K(v_p) \cap Q_{kw})$ and create new path $p_2 = \langle v_1, \ldots, v_{p-1}, v'_p, \rangle$. Based on path dominance relationships, in new paths, we insert only non-dominated partial paths into the queue and pruned dominated path.

TABLE II
PARTIAL VALID PATH IN THE PRIORITY QUEUE

Step	Paths ($\langle route\rangle$ ($length$), ($binary$), (X-NN)
1	①$(\langle v_1\rangle(0), 000, 0)$
2 dequeue ①	② $(\langle v_1, v_5\rangle(5), 001, 1)$
3 dequeue ②	③$(\langle v_1, v_5, v_3\rangle(17), 011, 1)$, ④ $(\langle v_1, v_2\rangle(6), 001, 2)$
4 dequeue ④	⑤$(\langle v_1, v_2, v_3\rangle(20), 011, 1)$, ⑥$(\langle v_1, v_3\rangle(8), 010, 3)$
5 dequeue ⑥	⑦$(\langle v_1, v_3, v_6\rangle(14), 110, 1)$, ⑧ $(\langle v_1, v_6\rangle(14), 100, 4)$
6 dequeue ⑦	⑨$(\langle v_1, v_3, v_6, v_9\rangle(21), 111, 1)$

Table II shows an example of *PruningKOSR*, Given the Query $Q(v_1, v_{12}, \{kw_1, kw_2, kw_3\})$. First, a priority queue PQ is established to store path information. The initial queue path is $(\langle v_1\rangle(0), 000, 0)$. From step 3 to step 5, path with the shortest length is dequeued. It should be noted that path ⑤ is expanded from path ④. However, based on the dominance relationship, path ③ can satisfy the same keywords,i.e.,kw_1, kw_2, with a shorter path length, thus path ⑤ is pruned. This process continues until the path ⑨ is expanded that satisfies Q_{kw}, algorithm terminates. The optimal route is $p_{opt} = \langle v_1, v_3, v_6, v_9, v_{12}\rangle$.

2) A based:* In *PruningKOSR*, paths with shorter lengths are given higher priority and are expanded first. However, *PruningKOSR* does not consider whether the current path is oriented towards the destination. Inspired by A*, the *StarKOSR* algotrthm [7] guides paths using a heuristic function. The purpose of the heuristic function is to estimate the cost of the current partial path.

Given a current path p=$\langle v_1, \ldots, v_p\rangle$ and another path p^* that satisfies the remaining keywords $(Q_{kw} \setminus K(p))$. We have $\delta(v_p, t) \leq \omega(p^*)$. which means $\delta(v_p, t)$ is an admissible estimation of the cost of route p. Therefore,the estimated cost of p is $\omega(p) + \delta(v_p, t)$. In *StarKOSR*, paths with lower estimated costs are given higher priority and are expanded first.

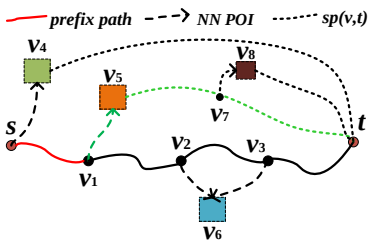

Fig. 5. Vertex Deviation Example

3) Vertex Deviation based: In the *EMB* algorithm, POIs near the s and t are prioritized and inserted into $sp(s,t)$. However, POIs selected by *DMD* strategy only consider the sum of distances from s to t, without considering whether the POIs is close to the actually shortest path $sp(s,t)$. Therefore, the vertex deviation based algorithm *DA-Prune* [5] starts from the actual shortest path and uses deviation strategy to find the nearest POI, thereby extending the path.

The main idea of *DA-Prune* is as follows: For example in Fig 5, *DA-Prune* first find $sp(s,t) = \langle s, v_1, v_2, v_3, t\rangle$, then sequentially performs vertex deviation from vertex on the $sp(s,t)$. Executes a vertex deviation strategy to find all the nearest POIs: v_4, v_5, v_6, v_7. For each valid path $p = \langle s, \ldots, vi\rangle$, the nearest of POI of v_i can satisfy at least a keyword in $Q'_{kw} = Q_{kw} \setminus K(p)$. It is worth noting that, the newly expanded paths can also be checked for dominance relationship in Definition 4, as the *NN* POI of different vertex might be the same one, e.g., v_2 and v_3 *NN* POI are both v_6. Then use a priority queue PQ to store all partial paths, e.g., $\langle s, v_1, v_5\rangle$, $\langle s, v_1, v_2, v_6\rangle$, $\langle s, v_1, v_2, v_3, v_6\rangle$. Since the vertices are deviated in sequence, they all share a common prefix path $\langle s, v_1\rangle$. Hence, *DA-Prune* builds the *ckPT* tree to store the concatenated paths of the prefix paths and their newly found *NN* POI and use $c(v_i)$ to denote the prefixed path from s to v_i, e.g., $c(v_2) = \langle s, v_1, v_2\rangle$. For each $c(v_i)$, we also check whether $sp(v_i,t)$ can satisfy $Q'_{kw} = Q_{kw} \setminus K(c(v_i))$. If satisfied Q'_{kw}, the path $\langle s, \ldots, v_i, \ldots, t\rangle$ is a feasible path,if not, then $c(v_i)$ is enqueued.

Subsequently, a path is selected from the PQ for expansion. *DA-Prune* incorporates both the current path distance and unsatisfied keywords, using $lb(c(v_i))$ to guide path expansion, $lb(c(v_i))$ is calculated by

$$lb(c(v_i)) = \omega(c(v_i)) + max_{LB} = \omega(c(v_i)) + \max_{\forall kw_m \in Q'_{kw}} (\min_{\forall v_j \in PSR(kw_m)} (\delta(v_i, v_j) + \delta(v_j, t))) \quad (4)$$

The goal of calculating $lb(c(v_i))$ is to evaluate the distance cost of paths expanding from $c(v_i)$. Thus, it is necessary to estimate the maximum distance cost max_LB of satisfying a remaining keyword. For each new prefixed path $c(v_i)$ in PQ, pop the one $c_{min}(v_i)$ with the minimum $lb(c(v_i))$ for expansion. In Fig 5, $lb(c(v_5))$ is minimum. So we continue by starting from the $sp(v_i,t)$ of c_{min}, iterating the aforementioned operations until a path that satisfies Q_{kw} is found.

Given the Query $Q(v_1, v_{10}, \{kw_1, kw_2, kw_3, kw_4\})$. We use Figure 6 to illustrate the process of the *DA-Prune* algorithm.

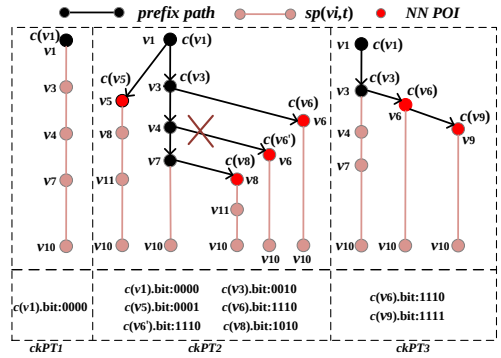

Fig. 6. ckPTs for $Q(v_1, v_{10}, \{kw_1, kw_2, kw_3, kw_4\})$ in Fig 2

First, we find the $sp(v_1, v_{10}) = \langle v_1, v_3, v_4, v_7, v_{10}\rangle$. Then, we start vertex deviation to find the nearest POIs, which are v_5, v_6, v_6, and v_8. Because the *NN* of v_3 and the *NN* of v_4 are both v_6, and $K(c(v'_6)) \cap Q_{kw} = K(c(v_6)) \cap Q_{kw}$, $\omega(c(v'_6)) > \omega(c(v'_6))$, therefore $c(v'_6)$ is pruned. At this time, three non-dominated partial paths are generated: $c(v_5)$, $c(v_6)$, and $c(v_8)$. In $ckPT_2$, each nearest POI as a leaf node of deviation vertex and the bitmaps of the covered keywords in the paths are also stored, e.g. $K(c(v_6)) = kw_2, kw_3, kw_4$, thus $c(v_6)$.bit=1110. After that, We compute $lb(c(v_i))$ and enqueued the path into priority queue PQ. By calculating $lb(c(v_i))$, $c(v_6)$ is dequeued for next round of deviation. Beacuse $sp(v_6, v_{10}) = v_6, v_{10}$, so only deviate from v_6 to find nearest POI, this POI is v_9. At this point, $c(v_9)$.bit=1111, which satisfied all keywords in Q_{kw}. Therefore, the algorithm terminates. The optimal route is $p_{opt} = \langle v_1, v_3, v_6, v_9, v_{10}\rangle$. $r = \{o_r, d_r, \Gamma_r, B_r, Soc_r\}$
.

V. EXPERIMENTS

A. Experimental Setup

Datasets: We use seven real-world road networks [12] with the corresponding POI data [22] as shown in Table **??**.

TABLE III
ROAD NETWORK

Map	$\|V\|$	$\|E\|$
CAL	21048	43386
NY	264636	733846
COL	435666	1057066
FLA	1070376	2712798
NE	1524253	3897636

TABLE IV
PARAMETERS

Parameter	Value
POI Density	0.02, 0.04, **0.08**, 0.16, 0.32
$\|K(V)\|$	**500**, 1000, 2000, 3000, 4000
$\|Q_{kw}\|$	2, 4, **6**, 8, 10
OD distance	0-10%, 10-20%, 20-30%, 30-40%, 40-50%
POI Distribution	**uniform**, Gaussian, , Clustered

REFERENCES

[1] N. Al Hasan Haldar, J. Li, M. Reynolds, T. Sellis, and J. X. Yu, "Location prediction in large-scale social networks: an in-depth benchmarking study," *The VLDB Journal*, vol. 28, pp. 623–648, 2019.

[2] E. H.-C. Lu, H.-S. Chen, and V. S. Tseng, "An efficient framework for multirequest route planning in urban environments," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 4, pp. 869–879, 2016.

[3] A. A. Haryanto, M. S. Islam, D. Taniar, and M. A. Cheema, "Ig-tree: an efficient spatial keyword index for planning best path queries on road networks," *World Wide Web*, vol. 22, pp. 1359–1399, 2019.

[4] J. Li, J. Hu, L. Li, X. Xiong, and X. Xia, "Efficient multi-request route planning on road network," in *2020 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*. IEEE, 2020, pp. 617–624.

[5] J. Li, X. Xiong, L. Li, D. He, C. Zong, and X. Zhou, "Finding top-k optimal routes with collective spatial keywords on road networks," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 368–380.

[6] T. Hashem and M. E. Ali, "Trip planning and scheduling queries in spatial databases: A survey," in *BDA*, ser. Lecture Notes in Computer Science, vol. 10721. Springer, 2017, pp. 164–178.

[7] H. Liu, C. Jin, B. Yang, and A. Zhou, "Finding top-k optimal sequenced routes," in *2018 IEEE 34th international conference on data engineering (ICDE)*. IEEE, 2018, pp. 569–580.

[8] E. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[10] R. Geisberger, P. Sanders, D. Schultes, and D. Delling, "Contraction hierarchies: Faster and simpler hierarchical routing in road networks," in *Experimental Algorithms: 7th International Workshop, WEA 2008 Provincetown, MA, USA, May 30-June 1, 2008 Proceedings 7*. Springer, 2008, pp. 319–333.

[11] D. Ouyang, L. Qin, L. Chang, X. Lin, Y. Zhang, and Q. Zhu, "When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks," in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 709–724.

[12] "9th dimacs implementation challenge - shortest paths," http://users.diag.uniroma1.it/challenge9/download.shtml.

[13] "source code," https://github.com/tenindra/RN-kNN-Exp.

[14] M. Sharifzadeh, M. Kolahdouzan, and C. Shahabi, "The optimal sequenced route query," *The VLDB journal*, vol. 17, pp. 765–787, 2008.

[15] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling-salesman problem," *Operations research*, vol. 21, no. 2, pp. 498–516, 1973.

[16] T. Hashem, T. Hashem, M. E. Ali, and L. Kulik, "Group trip planning queries in spatial databases," in *Advances in Spatial and Temporal Databases: 13th International Symposium, SSTD 2013, Munich, Germany, August 21-23, 2013. Proceedings 13*. Springer, 2013, pp. 259–276.

[17] T. Hashem, S. Barua, M. E. Ali, L. Kulik, and E. Tanin, "Efficient computation of trips with friends and families," in *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*, 2015, pp. 931–940.

[18] S. Samrose, T. Hashem, S. Barua, M. E. Ali, M. H. Uddin, and M. I. Mahmud, "Efficient computation of group optimal sequenced routes in road networks," in *2015 16th IEEE international conference on mobile data management*, vol. 1. IEEE, 2015, pp. 122–127.

[19] R. Zhong, G. Li, K.-L. Tan, L. Zhou, and Z. Gong, "G-tree: An efficient and scalable index for spatial search on road networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 8, pp. 2175–2189, 2015.

[20] R. Zhong, G. Li, K.-L. Tan, and L. Zhou, "G-tree: An efficient index for knn search on road networks," in *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*, 2013, pp. 39–48.

[21] B. Zheng, J. Xu, W.-C. Lee, and D. L. Lee, "Grid-partition index: a hybrid method for nearest-neighbor queries in wireless location-based services," *The VLDB Journal*, vol. 15, pp. 21–39, 2006.

[22] "Real datasets for spatial databases: Road networks and points of interest," http://www.cs.utah.edu/~lifeifei/SpatialDataset.htm.